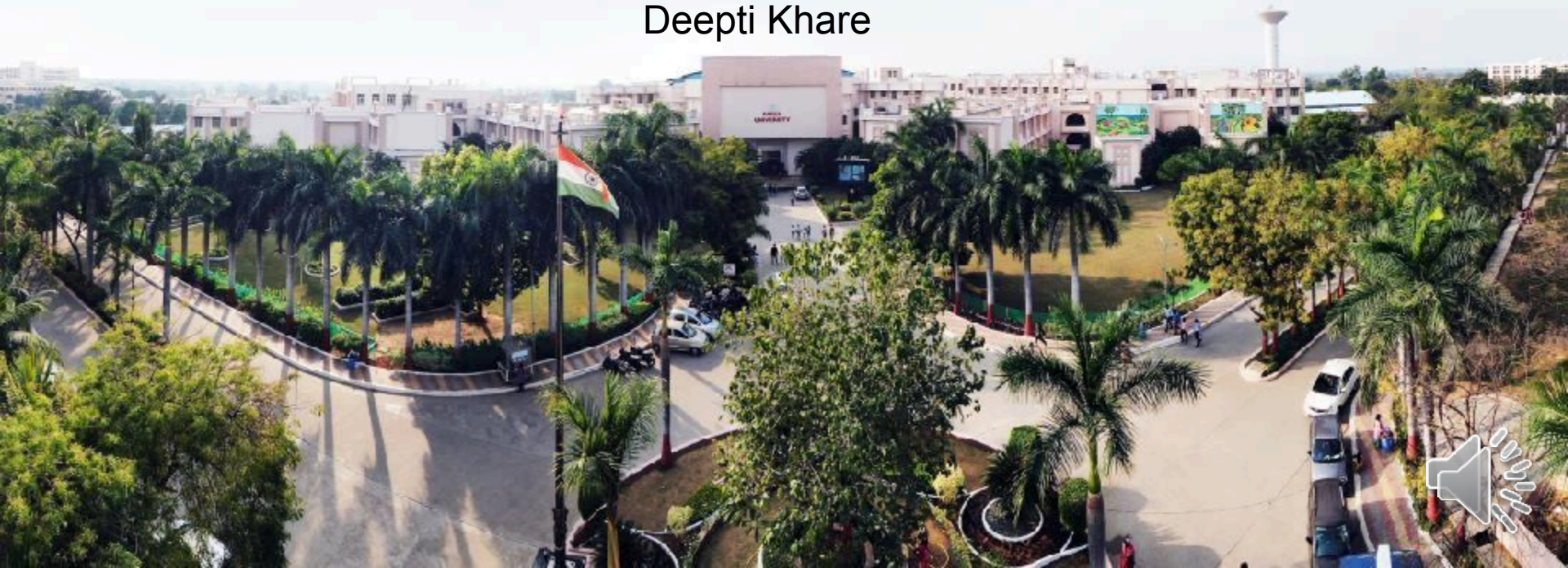




Full Stack Web Development

Created by :
Deepti Khare





CHAPTER-6

Angular JS





Angular JS

Introduction:

- **AngularJS** is a very powerful JavaScript Framework. It is used in Single Page Application (SPA) projects. AngularJS is open source, completely free, and used by thousands of developers around the world. It is licensed under the Apache license version 2.0.
- AngularJS is an open-source web application framework. It was originally developed in 2009 by Misko Hevery and Adam Abrons. It is now maintained by Google.
- AngularJS is a efficient framework that can create Rich Internet Applications (RIA).
- AngularJS provides developers an options to write client side applications using JavaScript in a clean Model View Controller (MVC) way.
- Applications written in Angular JS are cross-browser compliant. AngularJS automatically handles JavaScript code suitable for each browser.



Introduction conti.

AngularJS is a **JavaScript framework**. It can be added to an HTML page with a `<script>` tag. AngularJS extends HTML attributes with **Directives**, and binds data to HTML with **Expressions**.

```
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js">
</script>
```

AngularJS Extends HTML

AngularJS extends HTML with **ng-directives**.

The **ng-app** directive defines an AngularJS application.

The **ng-model** directive binds the value of HTML controls (input, select, textarea) to application data.

The **ng-bind** directive binds application data to the HTML view.

Introduction conti.

AngularJS Example

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js">
</script>
<body>

<div ng-app="">
  <p>Name: <input type="text" ng-model="name"></p>
  <p ng-bind="name"></p>
</div>

</body>
</html>
```

Example explained:

AngularJS starts automatically when the web page has loaded.

The **ng-app** directive tells AngularJS that the <div> element is the "owner" of an AngularJS **application**.

The **ng-model** directive binds the value of the input field to the application variable **name**.

The **ng-bind** directive binds the content of the <p> element to the application variable **name**.

Download Setup

When you open the link <https://angularjs.org/>, you will see there are two options to download AngularJS library –

PU

Example

```
<!doctype html>
<html>
  <head>
    <script src =
"https://ajax.googleapis.com/ajax/libs/angularjs/1.5.2/angular.min.js"></script>
  </head>

  <body ng-app = "myapp">
    <div ng-controller = "HelloController" >
      <h2>Welcome {{helloTo.title}} to the world of
Tutorialspoint!</h2>
    </div>
    <script>
      angular.module("myapp", [])

      .controller("HelloController", function($scope) {
        $scope.helloTo = {};
        $scope.helloTo.title = "AngularJS";
      });
    </script>

  </body>
</html>
```

Include AngularJS

We include the AngularJS JavaScript file in the HTML page so that we can use it –

```
<head>
  <script src =
"https://ajax.googleapis.com/ajax/libs/angularjs/1.4.8/angular.min.js">
</script>
</head>
```

Point to AngularJS app

Next, it is required to tell which part of HTML contains the AngularJS app. You can do this by adding the ng-app attribute to the root HTML element of the AngularJS app. You can either add it to the html element or the body element as shown below –

```
<body ng-app = "myapp">
</body>
```

The view is this part –

```
<div ng-controller = "HelloController" >
  <h2>Welcome {{helloTo.title}} to the world of Tutorialspoint!
</h2>
</div>
```

ng-controller tells AngularJS which controller to use with this view. helloTo.title tells AngularJS to write the model value named helloTo.title in HTML at this location.

What happens when the page is loaded in the browser ? Let us see

1. HTML document is loaded into the browser, and evaluated by the browser.
2. AngularJS JavaScript file is loaded, the angular *global* object is created.
3. The JavaScript which registers controller functions is executed.
4. Next, AngularJS scans through the HTML to search for AngularJS apps as well as views.
5. Once the view is located, it connects that view to the corresponding controller function.
6. Next, AngularJS executes the controller functions.
7. It then renders the views with data from the model populated by the controller. The page is now ready.

MVC Architecture

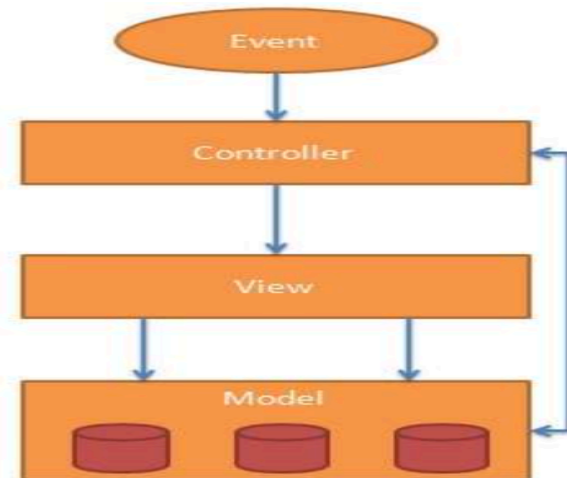
Model **V**iew **C**ontroller or MVC as it is popularly called, is a software design pattern for developing web applications. A Model View Controller pattern is made up of the following three parts –

Model – It is the lowest level of the pattern responsible for maintaining data.

View – It is responsible for displaying all or a portion of the data to the user.

Controller – It is a software Code that controls the interactions between the Model and View.

MVC is popular because it isolates the application logic from the user interface layer and supports separation of concerns. The controller receives all requests for the application and then works with the model to prepare any data needed by the view. The view then uses the data prepared by the controller to generate a final presentable response. The MVC abstraction can be graphically represented as follows.



MVC Architecture conti.

The Model:

The model is responsible for managing application data. It responds to the request from view and to the instructions from controller to update itself.

The View:

A presentation of data in a particular format, triggered by the controller's decision to present the data. They are script-based template systems such as JSP, ASP, PHP and very easy to integrate with AJAX technology.

The Controller:

The controller responds to user input and performs interactions on the data model objects. The controller receives input, validates it, and then performs business operations that modify the state of the data model.

AngularJS is a MVC based framework. In the coming chapters, we will see how AngularJS uses MVC methodology.

FirstProg (Testangular.html)

```
<html>
  <head>
    <title>AngularJS First Application</title>
  </head>
  <body>
    <h1>Sample Application</h1>
    <div ng-app = "">
      <p>Enter your Name: <input type = "text" ng-
model = "name"></p>
      <p>Hello <span ng-bind = "name"></span>!</p>
    </div>
    <script src = "https://ajax.googleapis.com/ajax/libs/
angularjs/1.3.14/angular.min.js">
    </script>
  </body>
</html>
```

Sample Application

Enter your Name:

Hello ede!

Angular JS Expressions

Expressions in AngularJS are used to bind application data to HTML. The expressions are resolved by AngularJS and the result is returned back to where the expression is written. The expressions in AngularJS are written in double braces: {{ expression }}. They behave similar to ng-bind directives: ng-bind="expression".

Syntax:

{{ expression }}

```
<!DOCTYPE html>
<html>

<head>
  <script src=
"https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js">
  </script>
  <title>AngularJS Expression</title>
</head>

<body style="text-align:center">
  <h1 style="color:green">FSW Class</h1>
  <h3>AngularJS Expressions</h3>
  <div ng-app=""
    ng-init="name='Hello PG Sem 3 Students'">
    <p> {{ name }} Welcome to the FSW Class.</p>
  </div>
</body>
</html>
```

FSW Class

AngularJS Expressions

Hello PG Sem 3 Students Welcome to the FSW Class.

Expression conti.

Example 2: This example displays the content of the object that we feed in the [ng-init directive](#).

```
<!DOCTYPE html>
<html>

<head>
  <script src=
    "https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js">
  </script>
  <title>AngularJS Expression</title>
</head>

<body>
  <h3>AngularJS Expressions</h3>
  <div ng-app=""
    ng-init="sort={one:'quick sort',
      two:'merge sort',
      three:'bubble sort'}">

    <p> Sorting Algorithms:</p>
    <ul>
      <li>{{ sort.one }}</li>
      <li>{{ sort.two }}</li>
      <li>{{ sort.three }}</li>
    </ul>
  </div>
</body>
</html>
```

AngularJS Expressions

Sorting Algorithms:

- quick sort
- merge sort
- bubble sort

Expression 3

This example illustrates the implementation of the objects with the Expression in AngularJS.

```
<!DOCTYPE html>
<html>

<head>
  <script src=
"https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js">
  </script>
</head>

<body>
  <center>
    <h3 style="color:green">
      AngularJS Expressions
    </h3>
    <div ng-app="" ng-init="data={Course:'AngularJS',
      topic:'Expression'}">
      <p>This is
        <span ng-bind="data.Course"></span>
        <span ng-bind="data.topic"></span>
        Tutorial
      </p>
    </div>
  </center>
</body>

</html>
```

AngularJS Expressions

This is AngularJS Expression Tutorial

Modules

An AngularJS module defines an application.

The module is a container for the different parts of an application.

The module is a container for the application controllers.

Controllers always belong to a module.

Creating a Module

A module is created by using the AngularJS function `angular.module`

The "myApp" parameter refers to an HTML element in which the application will

```
<div ng-app="myApp">...</div>

<script>

var app = angular.module("myApp", []);

</script>
```

Modules conti.

Adding a Controller:

Add a controller to your application, and refer to the controller with the ng-controller directive:

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js">
</script>
<body>

<div ng-app="myApp" ng-controller="myCtrl">
  {{ firstName + " " + lastName }}
</div>

<script>
var app = angular.module("myApp", []);
app.controller("myCtrl", function($scope) {
  $scope.firstName = "Deepti";
  $scope.lastName = "Shrivastava";
});
</script>

</body>
</html>
```

Deepti Shrivastava

AngularJS Directives

- AngularJS lets you extend HTML with new attributes called **Directives**.
- AngularJS has a set of built-in directives which offers functionality to your applications.
- AngularJS also lets you define your own directives.
- AngularJS directives are extended HTML attributes with the prefix ng-.

The ng-app Directive:

The ng-app directive defines the root element of an AngularJS application.

The ng-app directive will auto-bootstrap (automatically initialize) the application when a web page is loaded.

The ng-init Directive

The ng-init directive defines initial values for an AngularJS application.

Normally, you will not use ng-init. You will use a controller or module instead.

The ng-model Directive

The ng-model directive binds the value of HTML controls (input, select, textarea) to application data.

Angular Directives Example

```
<!DOCTYPE html>
<html>
<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js"></script>
<body>

<div data-ng-app="" data-ng-init="quantity=1;price=5">

<h2>Cost Calculator</h2>

Quantity: <input type="number" ng-model="quantity">
Price: <input type="number" ng-model="price">

<p><b>Total in dollar:</b> {{quantity * price}}</p>

</div>

</body>
</html>
```

Cost Calculator

Quantity: Price:

Total in dollar: 10



Angular ng-repeat Directive:

The **ng-repeat** directive repeats an HTML element:

The ng-repeat directive actually clones HTML elements once for each item in a collection.

```
<!DOCTYPE html>
<html>
<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js"
></script>
<body>

<div ng-app="" ng-init="names=['Jani','Hege','Kai']">
  <p>Looping with ng-repeat:</p>
  <ul ng-repeat="x in names">
    {{x}}
  </ul>
</div>

</body>
</html>
```

Looping with ng-repeat:

Jani

Hege

Kai

AngularJS Data Binding

Data binding in AngularJS is the synchronization between the model and the view.

Data Model:

AngularJS applications usually have a data model. The data model is a collection of data available for the application.

Example

```
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope) {
    $scope.firstname = "John";
    $scope.lastname = "Doe";
});
```

HTML View :The HTML container where the AngularJS application is displayed, is called the view. The view has access to the model, and there are several ways of displaying model data in the view. You can use the ng-bind directive, which will bind the innerHTML of the element to the specified model property:

```
<p ng-bind="firstname"></p>
```


AngularJS Data Binding conti.

You can also use double braces `{{ }}` to display content from the model:

```
<p>First name: {{firstname}}</p>
```

Or you can use the `ng-model` directive on HTML controls to bind the model to the view.

Use the `ng-model` directive to bind data from the model to the view on HTML controls (input, select, textarea)

Two-way Binding:

Data binding in AngularJS is the synchronization between the model and the view. When data in the *model* changes, the *view* reflects the change, and when data in the *view* changes, the *model* is updated as well. This happens immediately and automatically, which makes sure that the model and the view is updated at all times.



Two Way Binding Example

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js"></script>
<body>

<div ng-app="myApp" ng-controller="myCtrl">
  Name: <input ng-model="firstname">
  <h1>{{firstname}}</h1>
</div>

<script>
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope) {
  $scope.firstname = "John";
  $scope.lastname = "Doe";
});
</script>

<p>Change the name inside the input field, and the model data will change automatically, and
therefore also the header will change its value.</p>

</body>
</html>
```

Name:

John

Change the name inside the input field, and the model data will change automatically, and therefore also the header will change its value.

Controllers

AngularJS controllers control the data of AngularJS applications.

AngularJS applications are controlled by controllers.

The ng-controller directive defines the application controller.

A controller is a JavaScript Object, created by a standard JavaScript object constructor.

Parul



Controllers Example

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js"></script>
<body>
<div ng-app="myApp" ng-controller="myCtrl">
First Name: <input type="text" ng-model="firstName"><br>
Last Name: <input type="text" ng-model="lastName"><br>
<br>
Full Name: {{firstName + " " + lastName}}
</div>
<script>
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope) {
    $scope.firstName = "John";
    $scope.lastName = "Doe";
});
</script>
</body>
</html>
```

Application explained:

The AngularJS application is defined by `ng-app="myApp"`. The application runs inside the `<div>`.

The `ng-controller="myCtrl"` attribute is an AngularJS directive. It defines a controller. The `myCtrl` function is a JavaScript function. AngularJS will invoke the controller with a `$scope` object.

In AngularJS, `$scope` is the application object (the owner of application variables and functions).

The controller creates two properties (variables) in the scope (`firstName` and `lastName`).

The `ng-model` directives bind the input fields to the controller properties (`firstName` and `lastName`).

Controller Method

The example above demonstrated a controller object with two properties: lastName and firstName.

A controller can also have methods (variables as functions):

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js"></script>
<body>
<div ng-app="myApp" ng-controller="personCtrl">
First Name: <input type="text" ng-model="firstName"><br>
Last Name: <input type="text" ng-model="lastName"><br>
<br>
Full Name: {{fullName()}}
</div>
<script>
var app = angular.module('myApp', []);
app.controller('personCtrl', function($scope) {
    $scope.firstName = "John";
    $scope.lastName = "Doe";
    $scope.fullName = function() {
        return $scope.firstName + " " + $scope.lastName;
    };
});
</script>
</body>
</html>
```

First Name:

Last Name:

Full Name: John Doe

Controllers In External Files

In larger applications, it is common to store controllers in external files.



AngularJS Scope

The scope is the binding part between the HTML (view) and the JavaScript (controller).
The scope is an object with the available properties and methods.
The scope is available for both the view and the controller.

How to Use the Scope?

When you make a controller in AngularJS, you pass the \$scope object as an argument:
Properties made in the controller, can be referred to in the view:

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js">
</script>
<body>
<div ng-app="myApp" ng-controller="myCtrl">
<h1>{{carname}}</h1>
</div>
<script>
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope) {
    $scope.carname = "Volvo";
});
</script>
<p>The property "carname" was made in the controller, and can be referred to in the
view by using the {{ }} brackets.</p>
</body>
</html>
```

Volvo

The property "carname" was made in the controller, and can be referred to in the view by using the {{ }} brackets.

When adding properties to the \$scope object in the controller, the view (HTML) gets access to these properties. In the view, you do not use the prefix \$scope, you just refer to a property name, like {{carname}}.



Understanding the Scope:

If we consider an AngularJS application to consist of:

View, which is the HTML. **Model**, which is the data available for the current view.

Controller, which is the JavaScript function that makes/changes/removes/controls the data.

Then the scope is the Model.

The scope is a JavaScript object with properties and methods, which are available for both the view and the controller.

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js">
</script>
<body>
<div ng-app="myApp" ng-controller="myCtrl">
<input ng-model="name">
<h1>My name is {{name}}</h1>
</div>
<script>
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope) {
    $scope.name = "John Doe";
});
</script>
<p>When you change the name in the input field, the changes will affect the model,
and it will also affect the name property in the controller.</p>
</body>
</html>
```

John Doe

My name is John Doe

When you change the name in the input field, the changes will affect the model, and it will also affect the name property in the controller.

Know Your Scope

It is important to know which scope you are dealing with, at any time.

In the two examples above there is only one scope, so knowing your scope is not an issue, but for larger applications there can be sections in the HTML DOM which can only access certain scopes.

Each `<li>` element has access to the current repetition object, in this case a string, which is referred to by using `x`.

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js">
</script>
<body>
<div ng-app="myApp" ng-controller="myCtrl">
<ul>
  <li ng-repeat="x in names">{{x}}</li>
</ul>
</div>
<script>
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope) {
  $scope.names = ["Emil", "Tobias", "Linus"];
});
</script>
<p>The variable "x" has a different value for each repetition, proving that each
repetition has its own scope.</p>
</body>
</html>
```

- Emil
- Tobias
- Linus

The variable "x" has a different value for each repetition, proving that each repetition has its own scope.



Root Scope

All applications have a `$rootScope` which is the scope created on the HTML element that contains the `ng-app` directive.

The `rootScope` is available in the entire application.

If a variable has the same name in both the current scope and in the `rootScope`, the application uses the one in the current scope.

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js"></script>
<body ng-app="myApp">
<p>The rootScope's favorite color:</p>
<h1>{{color}}</h1>
<div ng-controller="myCtrl">
<p>The scope of the controller's favorite color:</p>
<h1>{{color}}</h1>
</div>
<p>The rootScope's favorite color is still:</p>
<h1>{{color}}</h1>
<script>var app = angular.module('myApp', []);
app.run(function($rootScope) {
    $rootScope.color = 'blue';
});
app.controller('myCtrl', function($scope) {
    $scope.color = "red";
});</script>
<p>Notice that controller's color variable does not overwrite the rootScope's color value.</p>
</body>
</html>
```

The rootScope's favorite color:

blue

The scope of the controller's favorite color:

red

The rootScope's favorite color is still:

blue

Notice that controller's color variable does not overwrite the rootScope's color value.



AngularJS - Dependency Injection

Dependency Injection is a software design in which components are given their dependencies instead of hard coding them within the component. It relieves a component from locating the dependency and makes dependencies configurable. It also helps in making components reusable, maintainable and testable.

AngularJS provides a supreme Dependency Injection mechanism. It provides following core components which can be injected into each other as dependencies.

Value, Factory, Service, Provider, Constant

Value

Value is a simple JavaScript object, which is required to pass values to the controller during config phase (config phase is when AngularJS bootstraps itself).

Factory

Factory is a function which is used to return value. It creates a value on demand whenever a service or a controller requires it. It generally uses a factory function to calculate and return the value.

AngularJS - Dependency Injection conti.

Service

Service is a singleton JavaScript object containing a set of functions to perform certain tasks. Service is defined using `service()` function and it is then injected into the controllers.

Provider

Provider is used by AngularJS internally to create services, factory, etc. during the config phase. The following script can be used to create `MathService` that we created earlier. Provider is a special factory method with `get()` method which is used to return the value/service/factory.

Constant

Constants are used to pass values at the config phase considering the fact that value cannot be used during the config phase.

AngularJS - Dependency Injection conti.

```

1 <html>
2 <head>
3 </head>
4 </head>
5 <body>
6 <h2>AngularJS Sample Application</h2>
7
8 <div ng-app = "mainApp" ng-controller = "CalcController">
9   <p>Enter a number: <input type = "number" ng-model = "number" /></p>
10   <button ng-click = "square()">X<sup>2</sup></button>
11   <p>Result: {{result}}</p>
12 </div>
13
14 <script src = "https://ajax.googleapis.com/ajax/libs/angularjs/1.3.14/angular.min.js">
15 </script>
16 <script>
17   var mainApp = angular.module("mainApp", []);
18
19   mainApp.config(function($provide) {
20     $provide.provider('MathService', function() {
21       this.$get = function() {
22         var factory = {};
23
24         factory.multiply = function(a, b) {
25           return a * b;
26         }
27         return factory;
28       };
29     });
30   });

```

```

mainApp.value("defaultInput", 5);

mainApp.factory('MathService', function() {
  var factory = {};

  factory.multiply = function(a, b) {
    return a * b;
  }
  return factory;
});

mainApp.service('CalcService', function(MathService) {
  this.square = function(a) {
    return MathService.multiply(a,a);
  }
});

mainApp.controller('CalcController', function($scope, CalcService, defaultInput) {
  $scope.number = defaultInput;
  $scope.result = CalcService.square($scope.number);

  $scope.square = function() {
    $scope.result = CalcService.square($scope.number);
  }
});
</script>

</body>

```

AngularJS - Dependency Injection conti.

AngularJS Sample Application

Enter a number:

x^2

Result: 36

Angular JS filters

Filters can be added in AngularJS to format data.

Filter	Function
currency	Formats a number to currency format
date	Formats a date to a readable format
filter	Filters arrays based on conditions
json	Converts objects into JSON string
limitTo	Limits string/array to specified length
lowercase	Converts string to lowercase
uppercase	Converts string to uppercase
number	Formats number with decimal points
orderBy	Sorts array items by field or condition





Angular JS filters Ex.

```

<!DOCTYPE html>
<html>
<head>
  <title>AngularJS Filters Demo</title>
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body ng-app="myApp" ng-controller="filterCtrl">

  <h2>1. Uppercase & Lowercase</h2>
  <p>Original: {{ name }}</p>
  <p>Uppercase: {{ name | uppercase }}</p>
  <p>Lowercase: {{ name | lowercase }}</p>

  <h2>2. Currency & Number</h2>
  <p>Price: {{ price | currency }}</p>
  <p>Formatted Number: {{ price | number:2 }}</p>

  <h2>3. Date</h2>
  <p>Today: {{ today | date:'fullDate' }}</p>

  <h2>4. JSON</h2>
  <pre>{{ person | json }}</pre>

  <h2>5. limitTo</h2>
  <p>First 4 letters of name: {{ name | limitTo:4 }}</p>

  <h2>6. filter</h2>
  <input type="text" ng-model="searchText" placeholder="Search country">
  <ul>

```

```

</li></ul>
<h2>7. orderBy</h2>
<button ng-click="sortBy('name')">Sort by Name</button>
<button ng-click="sortBy('country')">Sort by Country</button>
<ul>
  <li ng-repeat="x in people | orderBy:sortField">
    {{ x.name }} - {{ x.country }}
  </li>
</ul>
<script>
  angular.module('myApp', []).controller('filterCtrl', function($scope) {
    $scope.name = "AngularJSFilterDemo";
    $scope.price = 1234.567;
    $scope.today = new Date();
    $scope.person = { name: "Alex", age: 30, city: "New York" };
    $scope.people = [
      { name: "Jani", country: "Norway" },
      { name: "Carl", country: "Sweden" },
      { name: "Margareth", country: "England" },
      { name: "Joe", country: "Denmark" },
      { name: "Kai", country: "Norway" }
    ];

    $scope.sortField = 'name';

```

AngularJS - Tables

Table data is generally repeatable. The ng-repeat directive can be used to draw table easily. The following example shows the use of ng-repeat directive to draw a table –

```
<table>
  <tr>
    <th>Name</th>
    <th>Marks</th>
  </tr>

  <tr ng-repeat = "subject in student.subjects">
    <td>{{ subject.name }}</td>
    <td>{{ subject.marks }}</td>
  </tr>
</table>
```

Table can be styled using CSS Styling.

```
<style>
  table, th , td {
    border: 1px solid grey;
    border-collapse: collapse;
    padding: 5px;
  }
  table tr:nth-child(odd) {
    background-color: #f2f2f2;
  }
  table tr:nth-child(even) {
    background-color: #ffffff;
  }
</style>
```

AngularJS - HTML DOM

A Data Object Model (DOM), in the context of software development, refers to a set of classes or a structured way to represent and manage data within an application, particularly for complex processing and scenarios. It acts as an interface between the application data and its logic, providing a way to access, manipulate, and store data. The following directives are used to bind application data to the attributes of HTML DOM elements –

Sr.No.	Name & Description
1	ng-disabled disables a given control.
2	ng-show shows a given control.
3	ng-hide hides a given control.
4	ng-click represents a AngularJS click event.

DOM - directives

ng-disabled Directive

Add ng-disabled attribute to an HTML button and pass it a model. Bind the model to a checkbox and see the variation.

```
<input type = "checkbox" ng-model = "enableDisableButton">Disable Button  
<button ng-disabled = "enableDisableButton">Click Me!</button>
```

ng-show Directive

Add ng-show attribute to an HTML button and pass it a model. Bind the model to a checkbox and see the variation.

```
<input type = "checkbox" ng-model = "showHide1">Show Button  
<button ng-show = "showHide1">Click Me!</button>
```

DOM - directives

ng-hide Directive

Add ng-hide attribute to an HTML button and pass it a model. Bind the model to a checkbox and see the variation.

```
<input type = "checkbox" ng-model = "showHide2">Hide Button  
<button ng-hide = "showHide2">Click Me!</button>
```

ng-click Directive

Add ng-click attribute to an HTML button and update a model. Bind the model to HTML and see the variation.

```
<p>Total click: {{ clickCounter }}</p>  
<button ng-click = "clickCounter = clickCounter + 1">Click Me!</button>
```

Angular JS Forms

AngularJS enriches form filling and validation. We can use ng-click event to handle the click button and use \$dirty and \$invalid flags to do the validation in a seamless way. Use novalidate with a form declaration to disable any browser-specific validation. The form controls make heavy use of AngularJS events. Let us have a look at the events first.

Events

AngularJS provides multiple events associated with the HTML controls. For example, ng-click directive is generally associated with a button. AngularJS supports the following events –

- ng-click
- ng-dbl-click
- ng-mousedown
- ng-mouseup
- ng-mouseenter
- ng-mouseleave
- ng-mousemove
- ng-mouseover
- ng-keydown
- ng-keyup
- ng-keypress
- ng-change

Angular JS Forms

```
<!DOCTYPE html>
<html lang="en">
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js"></script>
<body>

<div ng-app="myApp" ng-controller="formCtrl">
  <form novalidate>
    First Name:<br>
    <input type="text" ng-model="user.firstName"><br>
    Last Name:<br>
    <input type="text" ng-model="user.lastName">
    <br><br>
    <button ng-click="reset()">RESET</button>
  </form>
  <p>form = {{user}}</p>
  <p>master = {{master}}</p>
</div>

<script>
var app = angular.module('myApp', []);
app.controller('formCtrl', function($scope) {
  $scope.master = {firstName:"John", lastName:"Doe"};
  $scope.reset = function() {
    $scope.user = angular.copy($scope.master);
  };
  $scope.reset();
});
</script></body></html>
```

First Name:

Last Name:

form = {"firstName":"John","lastName":"Doe"}

master = {"firstName":"John","lastName":"Doe"}

Form Validations

What is Form Validation?

Form validation ensures that the user enters correct and complete information in form fields before submission. AngularJS provides built-in support for form validation using HTML5 attributes, AngularJS directives, and CSS classes.

Key Directives Used in AngularJS Form Validation:

Directive	Description
ng-model	Binds input fields to a model
ng-required	Marks a field as required
ng-minlength	Specifies the minimum number of characters allowed
ng-maxlength	Specifies the maximum number of characters allowed
ng-pattern	Validates the input against a regex pattern
name	Assigns a name to input/form for reference in validation
novalidate	Disables native HTML5 validation (used in <form>)

Built-in Validation Properties (Form & Input Level)

Property	Description
\$valid	Field is valid
\$invalid	Field is invalid
\$touched	Field has been focused and then blurred
\$untouched	Field has not been focused
\$dirty	Field has been modified
\$pristine	Field has not been modified

Form Validation example

```
<form name="userForm" novalidate>
  <label>Name:</label>
  <input type="text" name="userName" ng-model="user.name" required>
  <span ng-show="userForm.userName.$touched && userForm.userName.$invalid">
    Name is required.
  </span>

  <br><br>

  <label>Email:</label>
  <input type="email" name="userEmail" ng-model="user.email" required>
  <span ng-show="userForm.userEmail.$touched && userForm.userEmail.$invalid">
    Valid email is required.
  </span>

  <br><br>

  <button ng-disabled="userForm.$invalid">Submit</button>
</form>
```

Name: fchch

Name is required.

Email:

Valid email is required.

Submit

AngularJS Animations

What is an Animation?

An animation is when the transformation of an HTML element gives you an illusion of motion.

Applications should not be filled with animations, but some animations can make the application easier to understand.

To make your applications ready for animations, you must include the AngularJS Animate library:

```
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular-animate.js"></script>
```

Then you must refer to the ngAnimate module in your application:

```
<body ng-app="ngAnimate">
```

Or if your application has a name, add ngAnimate as a dependency in your application module:

```
<script>  
var app = angular.module('myApp', ['ngAnimate']);  
</script>
```

Example

```
users > deepi > OneDrive > Desktop > Java Prog > angularanimationhtml > html > body
<!DOCTYPE html>
<html>
<style>
div {
  transition: all linear 0.5s;
  background-color: lightblue;
  height: 100px;
  width: 100%;
  position: relative;
  top: 0;
  left: 0;
}
.ng-hide {
  height: 0;
  width: 0;
  background-color: transparent;
  top: -200px;
  left: 200px;
}
</style>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js"></script>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular-animate.js"></script>
<body ng-app="myApp">
<h1>Hide the DIV: <input type="checkbox" ng-model="myCheck"></h1>
<div ng-hide="myCheck"></div>
</script>
var app = angular.module('myApp', ['ngAnimate']);
</script></body></html>
```

Hide the DIV: ☐



CRUD operation.

Index.html

```
<!DOCTYPE html>
<html ng-app="crudApp">
<head>
  <meta charset="utf-8">
  <title>AngularJS CRUD Example</title>
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js"></script>
  <script src="angularCrud.js"></script>
</head>
<body ng-controller="myCtrl">
  <h2>Student Management</h2>
  <form ng-submit="saveRecord()">
    <input type="text" ng-model="newStudent.name" placeholder="Name" required>
    <input type="text" ng-model="newStudent.address" placeholder="Address" required>
```


CRUD conti

```
<input type="text" ng-model="newStudent.dept" placeholder="Department" required>
  <button type="submit">{{ editingId === null ? 'Add' : 'Update' }}</button>
  <button type="button" ng-click="reset()">Cancel</button>
</form>
<table border="1">
  <tr><th>Name</th><th>Address</th><th>Dept</th><th>Action</th></tr>
  <tr ng-repeat="s in students">
    <td>{{s.name}}</td>
    <td>{{s.address}}</td>
    <td>{{s.dept}}</td>
    <td>
      <button ng-click="edit(s.id)">Edit</button>
      <button ng-click="delete(s.id)">Delete</button>
    </td>
  </tr>
</table></body></html>
```

angularCrud.js

```
var app = angular.module('crudApp', []);

app.controller('myCtrl', function($scope) {
  $scope.students = [];
  $scope.newStudent = {};
  $scope.editingId = null;

  function resetNew() {
    $scope.newStudent = {};
    $scope.editingId = null;
  }

  $scope.saveRecord = function() {
    if ($scope.editingId === null) {
      var nextId = $scope.students.length ?
        Math.max(...$scope.students.map(s => s.id)) + 1 : 1;
      $scope.newStudent.id = nextId;
      $scope.students.push(angular.copy($scope.newStudent));
    } else
```

angularCrud.js conti

```
{  
  var idx = $scope.students.findIndex(s => s.id === $scope.editingId);  
  if (idx !== -1) {  
    $scope.students[idx] = angular.copy($scope.newStudent);  
  }  
}  
resetNew();  
};  
  
$scope.edit = function(id) {  
  var student = $scope.students.find(s => s.id === id);  
  if (student) {  
    $scope.newStudent = angular.copy(student);  
    $scope.editingId = id;  
  }  
};
```

angularCrud.js conti

```
$scope.delete = function(id) {  
  var idx = $scope.students.findIndex(s => s.id === id);  
  if (idx !== -1) $scope.students.splice(idx, 1);  
  if ($scope.editingId === id) resetNew();  
};  
  
$scope.reset = resetNew;  
});
```

Student Management

Name	Address	Department	Add	Cancel
Name	Address	Dept	Action	